

퍼스널컬러 진단 AI 구성(박성은)

📅 날짜	June 21, 2026
⚙️ 상태	진행 중
☰ 태그	
🕒 생성 일시	June 21, 2026 4:45 PM

응, 네가 생각한 큰 구조는 **방향 자체는 맞아**.

다만 그대로 “사용자 이미지 + 프롬프트 → AI가 전부 판단 → JSON 반환 → 이미지 생성”으로만 만들면 결과가 흔들릴 가능성이 커. 특히 퍼스널 컬러는 **조명, 화이트밸런스, 필터, 피부 보정, 배경색**에 영향을 많이 받기 때문에, AI에게 한 번에 다 맡기기보다는 **분석 파이프라인 + AI 보조 판단 + 고정 팔레트 매핑** 구조로 가는 게 더 안전해.

내가 보기엔 이 구조가 가장 적절해.

```
사용자 이미지 업로드
↓
이미지 품질 검사 / 전처리
↓
AI Vision 모델로 얼굴·피부톤·명도·채도·대비 분석
↓
진단 프롬프트 + 퍼스널컬러 기준표 + JSON Schema 전달
↓
AI가 diagnosis JSON 초안 반환
↓
백엔드에서 JSON 검증 + toneKey 정규화
↓
toneKey 기준으로 우리가 미리 만든 컬러칩 DB 매핑
↓
진단 결과 저장
↓
GenAI 이미지 생성 작업 비동기 실행
↓
result.vue에서 결과표 표시
```

핵심은 **AI가 컬러칩을 마음대로 많이 만들어내게 하지 않는 것**이야. AI가 `#F3B8C8`, `#D8B7DA` 같은 hex를 만들어줄 수는 있지만, 매번 조금씩 달라질 수 있어. 그러면 같은 `summer_cool_light` 결과인데 어떤 사람은 다른 팔레트가 뜨는 문제가 생겨. 그래서 더 좋은 방식은 **AI는 퍼스널 컬러**

러 타입과 분석값을 판단하고, 컬러칩은 우리 서비스가 미리 정의한 팔레트 테이블에서 가져오는 구조야.

결론부터 말하면

네 구조는 **Agent AI에 가깝다**기보다는 **"AI Orchestrated Workflow"**에 가까워.

Agent AI처럼 보이는 이유는 단계가 여러 개라서 그래.

진단 AI
→ JSON 생성
→ 이미지 생성 AI
→ 결과 페이지 반영

하지만 이건 AI가 스스로 판단해서 도구를 자유롭게 고르는 구조라기보다는, 우리가 정해둔 순서대로 실행하는 **고정 파이프라인**이야. 지금 프로젝트에서는 오히려 이게 더 좋아. Agent처럼 자율적으로 만들면 디버깅이 어렵고, 결과가 흔들리고, 왜 특정 결과가 나왔는지 설명하기 어려워져.

추천 구조

1. 이미지 업로드 단계

사용자가 이미지를 업로드하면 바로 AI에 넘기지 말고 먼저 검사하는 게 좋아.

확인할 것:

- 얼굴이 있는가
- 얼굴이 너무 작지 않은가
- 너무 어둡거나 밝지 않은가
- 필터/보정이 심하지 않은가
- 여러 명이 찍힌 사진은 아닌가
- 정면 얼굴인지
- 피부 영역을 분석할 수 있는지

이 단계에서 품질이 낮으면 바로 진단하지 말고 다시 업로드하게 하는 게 좋아.

예시 문구:

조명이 너무 어두워 정확한 진단이 어려워요.
자연광에서 정면 사진을 다시 업로드해주세요.

이걸 안 하면 AI가 억지로 결과를 만들 수 있어.

2. AI 진단 단계

여기서는 **Vision이 가능한 LLM**을 쓰는 게 맞아. 사용자 이미지를 보고, 우리가 만든 퍼스널 컬러 기준표와 함께 분석하도록 하는 방식이야. OpenAI 문서 기준으로 이미지 입력 분석과 이미지 생성은 별도 사용 사례로 나뉘며, 비전 모델은 이미지 입력을 분석할 수 있고 이미지 모델은 생성/편집에 사용된다.

하지만 AI에게 이렇게만 시키면 안 돼.

이 사람 퍼스널 컬러 진단해줘.

이건 너무 위험해.

대신 이렇게 줘야 해.

너는 Lumière의 퍼스널 컬러 진단 보조 모델이다.
다음 이미지를 분석하고, 아래 diagnosis schema에 맞춰서만 결과를 반환해라.
가능한 toneKey는 우리가 정의한 enum 중 하나만 선택해라.
확신이 낮거나 이미지 품질이 부족하면 low_confidence 또는 retry_needed를 반환해라.

그리고 반드시 **Structured Outputs / JSON Schema**를 써야 해. OpenAI 문서에서도 Structured Outputs는 모델 응답이 개발자가 정의한 JSON Schema를 따르도록 보장하는 기능이고, 일반 JSON mode보다 schema 준수까지 보장한다는 점을 설명하고 있어.

3. AI가 반환해야 하는 JSON

AI가 바로 `result.vue`에 들어갈 완성 데이터를 전부 만들게 하기보다는, 먼저 **진단 핵심 결과 JSON**을 반환하게 하는 게 좋아.

예시는 이런 식이야.

```
{
  "diagnosisStatus": "success",
  "toneKey": "summer_cool_light",
  "toneNameKo": "여름 쿨 라이트",
  "toneNameEn": "Summer Cool Light",
  "confidence": 0.87,
  "analysis": {
    "brightness": 65,
    "saturation": 30,
    "clarity": 82,
    "contrast": 35,
    "coolWarm": -62,
    "softness": 78
  },
  "reasonSummary": "밝고 맑은 피부톤, 낮은 채도, 부드러운 대비가 관찰되어 여름 쿨 라이트로 판단했습니다.",
  "imageQuality": {
    "isUsable": true,
    "issues": []
  },
  "recommendedProfileToneKey": "summer_cool_light",
  "makeoverPromptSeed": {
    "baseTone": "light cool pink beige",
    "eyeTone": "soft mauve, lavender pink, cool taupe",
    "lipTone": "clear rosy pink, berry pink",
    "blushTone": "light cool rose pink"
  }
}
```

여기서 중요한 건 **toneKey** 야.

toneKey 가 정해지면 백엔드에서 아래 것들을 안정적으로 붙여줄 수 있어.

- 결과별 기본 프로필 이미지
- BEST 컬러칩
- NEUTRAL 컬러칩
- ACCENT 컬러칩
- WORST 컬러칩
- 베이스 컬러 가이드
- 아이 메이크업 컬러 가이드
- 립 컬러 가이드
- 블러셔 컬러 가이드
- 스타일 가이드

즉, AI가 모든 컬러칩을 매번 새로 만들기보다, 우리의 **diagnosis form**에 맞는 고정 데이터와 결합하는 방식이 좋아.

컬러칩은 어떻게 처리하는 게 좋은가

이 부분은 네 프로젝트에서 매우 중요해.

비추천 구조

AI가 매번 BEST 컬러 hex 20개 생성
AI가 매번 립 컬러 hex 10개 생성
AI가 매번 아이새도우 색상 hex 생성

이러면 결과가 매번 달라질 수 있어.

같은 사람이 다시 진단했는데:

첫 번째: #F3B8C8
두 번째: #EFA9C2
세 번째: #F1B0CC

이런 식으로 미세하게 흔들릴 수 있음.

추천 구조

AI는 toneKey를 판단한다.
백엔드는 toneKey에 맞는 colorGuidePreset을 불러온다.

예를 들면:

```
const colorGuidePreset = {
  summer_cool_light: {
    best: [
      { name: "로지 핑크", hex: "#EFA6B8" },
      { name: "쿨 핑크", hex: "#E7A4C2" },
      { name: "라벤더 핑크", hex: "#D8B7DA" }
    ],
    eye: {
      highlighter: [
        { name: "실버 핑크 필", hex: "#F5DDEB" },
        { name: "라일락 필", hex: "#DCC9F0" }
      ]
    }
  }
}
```

```

    ],
    base: [
      { name:"페일 핑크", hex:"#F1C9D5" }
    ],
    shading: [
      { name:"소프트 모브 브라운", hex:"#A98791" }
    ],
    point: [
      { name:"쿨 로즈", hex:"#C77C98" }
    ]
  }
}
}

```

이렇게 해야 결과표가 안정적이고, 디자이너가 원하는 톤도 유지돼.

GenAI 이미지 생성은 분리하는 게 맞다

내가 말한 것처럼 진단 AI와 이미지 생성 AI는 분리하는 게 맞아.

진단 AI의 역할:

이미지를 보고 퍼스널 컬러 결과를 판단
 정량 지표 계산 보조
 toneKey 선택
 result JSON 생성

이미지 생성 AI의 역할:

진단 결과에 맞는 예상 메이크업 이미지 생성
 내추럴 / 데일리 / 청순 / 로맨틱 등 스타일 이미지 생성

OpenAI 이미지 문서 기준으로 이미지 생성/편집은 Image API 또는 Responses API에서 처리할 수 있고, 이미지 모델은 텍스트와 이미지 입력을 바탕으로 이미지를 생성하거나 편집할 수 있다.

다만 GenAI 이미지는 시간이 오래 걸릴 수 있으니까, 진단 결과 화면을 이미지 생성이 끝날 때까지 기다리게 만들 필요는 없어.

추천은 이거야.

1. 진단 JSON 먼저 생성
2. `result.vue` 먼저 표시
3. 기본 프로필 이미지는 `images/tones/`에서 즉시 표시
4. GenAI 메이크업 이미지는 "생성 중" 상태로 표시
5. 이미지 생성 완료 후 슬라이드에 추가

이렇게 하면 사용자는 결과를 빨리 볼 수 있고, 이미지 생성 실패해도 진단 결과표는 깨지지 않아.

파인튜닝은 지금 당장 필요 없을 가능성이 크다

현재 단계에서는 파인튜닝부터 생각하지 않는 게 맞아.

먼저 해야 할 건:

1. 퍼스널 컬러 타입 기준표 정리
2. `diagnosis JSON schema` 확정
3. `prompt` 작성
4. `mock` 데이터와 실제 이미지 몇 장으로 테스트
5. 결과가 얼마나 일관적인지 평가
6. 부족하면 `prompt` 수정
7. 그래도 부족하면 별도 모델/파인튜닝 검토

OpenAI 문서에서도 모델 최적화 흐름은 eval로 기준 성능을 측정하고, 프롬프트와 컨텍스트를 먼저 개선한 뒤, 필요한 경우 fine-tuning을 고려하는 순서로 설명한다. 현재 문서상 fine-tuning 플랫폼 접근성에도 제한이 있으므로, 프로젝트의 필수 구조로 잡기보다는 나중 단계로 두는 게 안전하다.

파인튜닝이 필요한 경우는 이런 때야.

- 전문가가 라벨링한 퍼스널 컬러 이미지 데이터가 충분히 있음
- 같은 이미지에 대해 모델이 계속 다른 `toneKey`를 반환함
- 프롬프트 개선만으로 정확도가 안 올라감
- 실제 서비스에서 누적된 진단/피드백 데이터가 있음
- "여름 쿨 라이트 vs 여름 쿨 뮤트" 같은 세부 분류가 계속 헷갈림

지금은 데이터셋이 충분하지 않을 가능성이 크기 때문에, **프롬프트 + JSON Schema + 고정 팔레트 매핑 + 평가 데이터셋**으로 먼저 가는 게 좋아.

내가 추천하는 최종 아키텍처

[Frontend]

diagnosis.vue

- 이미지 업로드
- 동의 체크
- 업로드 상태 표시

↓

[Backend]

POST /api/diagnosis/analyze

- 이미지 저장
- 이미지 품질 검사
- AI Vision 모델 호출
- Structured JSON 반환 받기
- JSON schema 검증
- toneKey 정규화

↓

[Diagnosis Resolver]

- toneKey 기준으로 colorGuidePreset 매핑
- images/tones 프로필 이미지 매핑
- result.vue용 최종 DiagnosisResult 생성

↓

[Database]

diagnosis_results

- userId
- toneKey
- confidence
- analysisJson
- colorGuideJson
- profileImageUrl
- createdAt

↓

[Async Image Job]

POST /api/diagnosis/:id/generate-makeover

- diagnosis result 기반 prompt 생성
- 이미지 생성 모델 호출
- 생성 이미지 저장
- makeoverImages 업데이트

↓

[Frontend]

result.vue

- 진단 결과 표시
- 기본 프로필 이미지 표시
- GenAI 이미지 생성 중 상태 표시
- 완료되면 슬라이드에 추가

중요한 설계 포인트

1. 프론트에서 AI API 키를 절대 노출하면 안 됨

AI 호출은 반드시 백엔드에서 해야 해.

Vue → 백엔드 API → AI API

이렇게 가야 함.

프론트에서 직접 AI API를 부르면 API 키가 노출될 수 있어.

2. result.vue는 AI 응답을 직접 믿으면 안 됨

AI가 JSON을 반환해도 바로 화면에 넣지 말고 백엔드에서 검증해야 해.

검증할 것:

- toneKey가 허용된 enum인가
- confidence가 0~1 범위인가
- brightness, saturation 값이 0~100인가
- hex 코드 형식이 맞는가
- 필수 필드가 누락되지 않았는가
- imageQuality.isUsable이 false면 결과를 보여주지 않는가

3. hex 코드는 AI 생성보다 preset 추천

AI에게 hex를 “많이 받아야 한다”는 생각은 이해되지만, 서비스 안정성만 보면 **preset DB가 더 좋다**.

AI가 반환하는 건:

```
toneKey
analysis values
reasonSummary
makeoverPromptSeed
```

백엔드가 붙이는 건:

```
color chips
makeup guide
style guide
profile image
```

이렇게 역할을 나누는 게 좋아.

4. GenAI 이미지는 비동기로

진단 결과와 이미지 생성을 한 요청에서 다 처리하면 너무 느려질 수 있어.

추천:

```
진단 결과: 즉시 표시
메이크업 이미지: 생성 중 → 완료 후 표시
```

실패 시:

```
AI 메이크오버 이미지를 생성하지 못했어요.
기본 프로필 이미지로 결과를 확인할 수 있어요.
```

정리

네가 생각한 구조는 방향이 맞아.

다만 더 안정적으로 만들려면 이렇게 바꾸는 게 좋아.

```
AI가 모든 걸 결정하는 구조
```

보다는

AI는 진단 판단
백엔드는 결과 검증
우리 DB는 컬러칩/가이드 제공
GenAI는 이미지 생성만 담당
result.vue는 최종 정규화된 결과만 표시

이 구조가 훨씬 좋아.

즉 최종 구조는:

사용자 이미지
→ 이미지 품질 검사
→ Vision AI 진단
→ JSON Schema 기반 diagnosis result
→ toneKey 기준 preset 매핑
→ 결과 저장
→ GenAI 이미지 생성
→ result.vue 표시

이렇게 잡으면 돼.